

UNIVERSIDAD POLITÉCNICA DE PACHUCA

Ingeniería en Tecnologías de la Información e Innovación Digital

DESARROLLO DE APLICACIONES MÓVILES

Reporte de Práctica

Implementación de Row y LazyRow en Jetpack Compose

Alumno: Daniel Jimenez Pluma

Grupo: TIID 04 01

Docente: Miguel Angel Montoya Cerro

Lugar: Zempoala, Hidalgo, México

Fecha: 9 de octubre de 2026

Índice general

1 Introducción **5**

1.1 Contextualización 5

1.2 Planteamiento del problema 5

1.3 Objetivo general 5

1.4 Objetivos específicos 5

1.5 Justificación 6

1.6 Estructura del documento 6

2 Marco teórico **6**

2.1 Android Studio 6

2.2 Kotlin 6

2.3 Jetpack Compose 7

2.4 Row 7

2.5 LazyColumn y LazyRow 7

2.6 Modifier 7

2.7 Arrangement 8

2.8 Text e Image 8

3 Desarrollo **8**

3.1 Creación del proyecto 8

3.2 Recursos de la aplicación 10

3.3 Importaciones y actividad principal 11

3.4 Creación de la función composable 11

3.5 Incorporación de los textos 12

3.6 Implementación de LazyRow 13

3.7 Pruebas de la aplicación 16

4 Análisis y argumentación **17**

5 Resultados **18**

6 Explicación del código **18**

6.1 Paquete e importaciones 18

6.2 Actividad principal 19
6.3 Función composable y LazyColumn 19
6.4 Imagen y textos 20
6.5 LazyRow 21

7 Elementos principales utilizados 22

8 Conclusiones 22

8.1 Aportes y limitaciones 22
8.2 Recomendaciones y proyecciones 23

9 Trabajo futuro 23

10 Anexos 23

10.1 Anexo A. Código completo 23
10.2 Anexo B. Evidencias adicionales 26

11 Referencias 26

Índice de ilustraciones

1 Selección de la plantilla Empty Activity. 9
2 Configuración inicial del proyecto. 10
3 Imagen potro.png dentro de la carpeta res/drawable. 10
4 Importaciones y clase MainActivity con setContent. 11
5 Función composable app() con LazyColumn e Image. 12
6 Código de los componentes Text. 13
7 Código del LazyRow dentro de un item de la LazyColumn. 14
8 Vista previa de la pantalla con el LazyRow. 15
9 Desplazamiento vertical de la pantalla en el emulador. 16
10 Desplazamiento horizontal del LazyRow y resultado final. 17

Índice de tablas

1 Elementos principales utilizados en la práctica 22

1. Introducción

1.1. Contextualización

El desarrollo de aplicaciones móviles requiere herramientas que permitan construir interfaces de usuario de manera organizada, funcional y adaptable a diferentes tamaños de pantalla. Dentro del desarrollo de aplicaciones Android, Jetpack Compose proporciona un enfoque declarativo para la creación de interfaces mediante el lenguaje Kotlin (Google, s. f.-a).

Una tarea frecuente al diseñar una interfaz es decidir cómo se acomodan los elementos en el espacio disponible. Compose ofrece contenedores para organizarlos de forma vertical, horizontal o apilada, y versiones con desplazamiento para listas.

1.2. Planteamiento del problema

Cuando los elementos de una fila ocupan más ancho del que tiene la pantalla, una fila convencional (`Row`) no ofrece desplazamiento por sí sola y el contenido excedente queda fuera del área visible. La pregunta que guía esta práctica es cómo organizar elementos de forma horizontal y permitir que el usuario pueda desplazarse hacia los lados cuando el contenido no cabe en pantalla. Para resolverla se utilizó `LazyRow`, integrado dentro de una `LazyColumn`.

1.3. Objetivo general

Implementar una interfaz de usuario con Jetpack Compose que utilice un `LazyRow` dentro de una `LazyColumn`, comprendiendo cómo organizar contenido de manera horizontal con desplazamiento dentro de una aplicación Android.

1.4. Objetivos específicos

- Crear un proyecto Android utilizando Android Studio, Kotlin y Jetpack Compose.
- Implementar una interfaz mediante funciones composables.
- Organizar una pantalla con `LazyColumn` que contenga una imagen y varios textos.
- Incorporar un `LazyRow` dentro de un elemento de la `LazyColumn`.
- Aplicar modificadores para controlar tamaño, espaciado y apariencia de los componentes.

- Utilizar `Arrangement` para configurar la distribución de los elementos en el eje horizontal.
- Comprobar el desplazamiento horizontal cuando el contenido supera el ancho de la pantalla.

1.5. Justificación

La práctica resulta relevante porque las filas con desplazamiento horizontal son un patrón frecuente en aplicaciones reales, por ejemplo en listas de categorías, etiquetas o carruseles de contenido. Conocer el funcionamiento de `Row` y `LazyRow` permite elegir la estructura adecuada según la cantidad y el comportamiento del contenido.

Además, el ejercicio permite relacionar los conceptos teóricos con una implementación funcional, observando directamente el efecto que tienen los modificadores y componentes de Compose sobre la interfaz.

1.6. Estructura del documento

El documento se organiza de la siguiente manera. Primero se presenta el marco teórico relacionado con Android Studio, Kotlin y Jetpack Compose. Después se describe el desarrollo de la práctica con las evidencias obtenidas, seguido del análisis, los resultados y la explicación del código por bloques. Finalmente se incluyen las conclusiones, el trabajo futuro, los anexos y las referencias.

2. Marco teórico

2.1. Android Studio

Android Studio es el entorno de desarrollo utilizado para la creación de aplicaciones Android. Permite crear proyectos, editar código, administrar recursos, compilar aplicaciones y realizar pruebas mediante emuladores o dispositivos físicos.

En esta práctica fue utilizado como herramienta principal para crear el proyecto, implementar el código Kotlin y visualizar la interfaz desarrollada mediante Jetpack Compose.

2.2. Kotlin

Kotlin es el lenguaje de programación utilizado para desarrollar la aplicación. Es compatible con el desarrollo Android y permite utilizar Jetpack Compose mediante funciones declarativas (JetBrains, s. f.).

2.3. Jetpack Compose

Jetpack Compose es un toolkit moderno para la construcción de interfaces de usuario en Android mediante un enfoque declarativo. La interfaz se define mediante funciones composables escritas en Kotlin, en lugar de archivos XML (Google, s. f.-a).

2.4. Row

El componente `Row` organiza sus elementos de manera horizontal, colocándolos uno al lado del otro (Google, s. f.-b). Sus elementos se componen de inmediato y el contenedor no incluye desplazamiento por sí mismo, por lo que es adecuado para pocos elementos que caben en el ancho de la pantalla.

2.5. LazyColumn y LazyRow

`LazyColumn` y `LazyRow` son listas con desplazamiento vertical y horizontal, respectivamente. Su característica principal es que solo componen y dibujan los elementos visibles en pantalla, lo que las hace adecuadas para contenido largo o de tamaño desconocido (Google, s. f.-c). Los elementos se agregan mediante bloques `item{}`.

En esta práctica, la `LazyColumn` funciona como contenedor principal de la pantalla y el `LazyRow` se coloca dentro de uno de sus elementos. Esto es posible porque sus direcciones de desplazamiento son distintas.

2.6. Modifier

Los modificadores permiten controlar diferentes características de los componentes, como tamaño, apariencia y espaciado. Se encadenan en orden y el orden afecta el resultado final. Durante la práctica se utilizaron principalmente:

- `fillMaxSize()`
- `fillMaxWidth()`
- `height()`
- `padding()`
- `background()`

2.7. Arrangement

`Arrangement` define cómo se distribuye el espacio entre los elementos en el eje principal del contenedor. En un `LazyRow`, el eje principal es el horizontal y se configura mediante el parámetro `horizontalArrangement`. En la práctica se utilizó `Arrangement.SpaceBetween`, que reparte el espacio sobrante entre los elementos.

2.8. Text e Image

El componente `Text` permite mostrar texto y puede configurarse mediante propiedades como tamaño de fuente, color y alineación. El componente `Image` permite mostrar imágenes; para cargar el recurso gráfico se empleó `painterResource()`, y el parámetro `contentDescription` describe la imagen para los lectores de pantalla.

3. Desarrollo

3.1. Creación del proyecto

El desarrollo comenzó con la creación de un nuevo proyecto en Android Studio utilizando Kotlin y Jetpack Compose. Se seleccionó la plantilla *Empty Activity*, que genera una actividad basada en Compose.

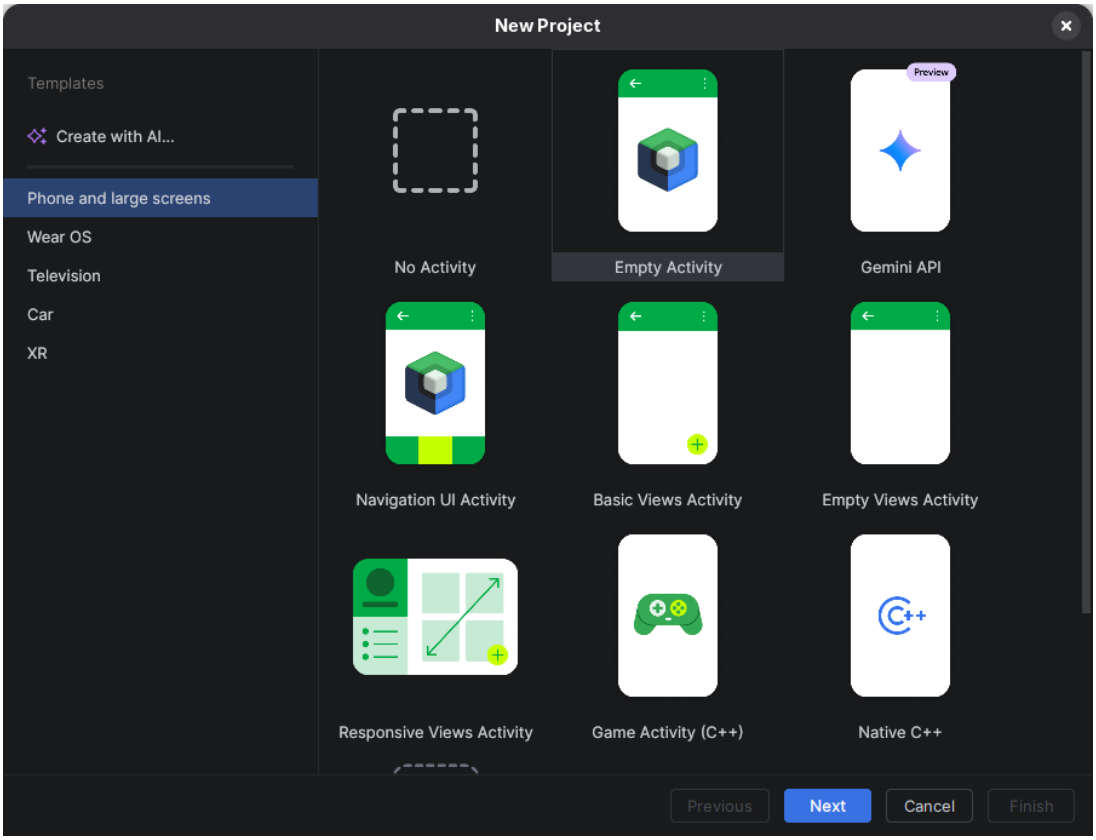


Ilustración 1: Selección de la plantilla Empty Activity.

Posteriormente se configuraron el nombre del proyecto (`ColumnAplication`), el paquete (`com.example.columnaplication`), el SDK mínimo y el lenguaje de configuración de compilación.

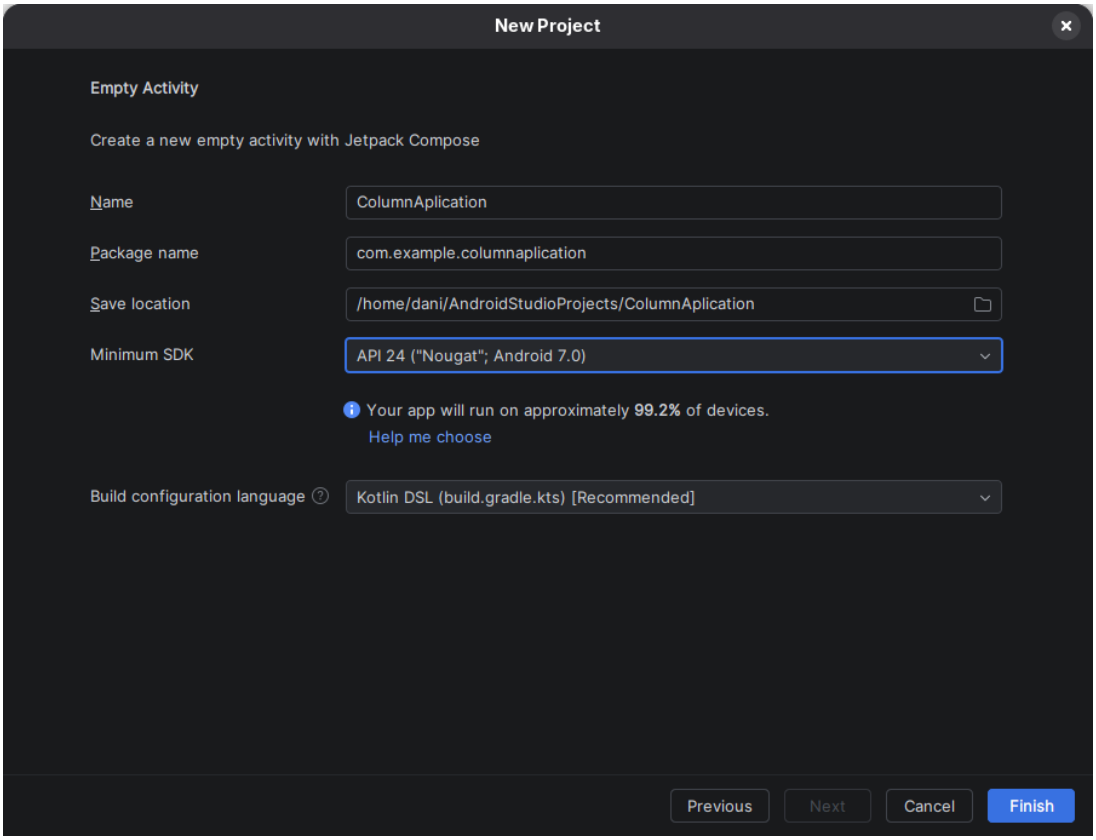


Ilustración 2: Configuración inicial del proyecto.

3.2. Recursos de la aplicación

Para mostrar la imagen se agregó el archivo `potro.png` a la carpeta `res/drawable` del proyecto, de modo que pudiera referenciarse como `R.drawable.potro`.

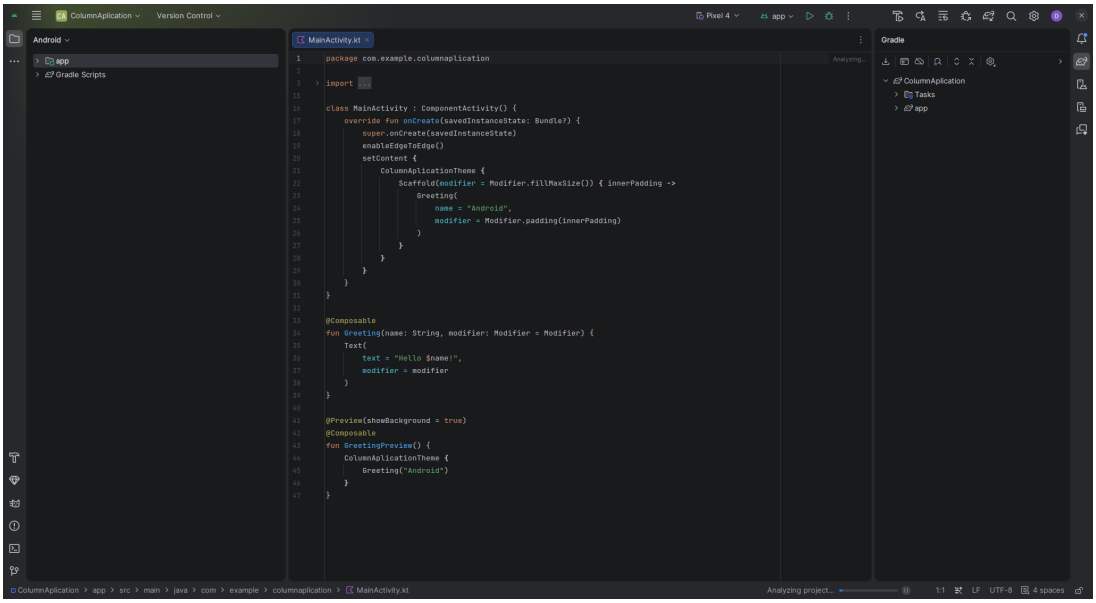


Ilustración 3: Imagen potro.png dentro de la carpeta res/drawable.

3.3. Importaciones y actividad principal

Se agregaron las importaciones necesarias, entre ellas `LazyColumn`, `LazyRow`, `Arrangement` y `padding`, además de las utilizadas para imagen, texto y modificadores. La actividad principal conserva el método `setContent`, que llama a la función `app()` dentro del tema del proyecto.

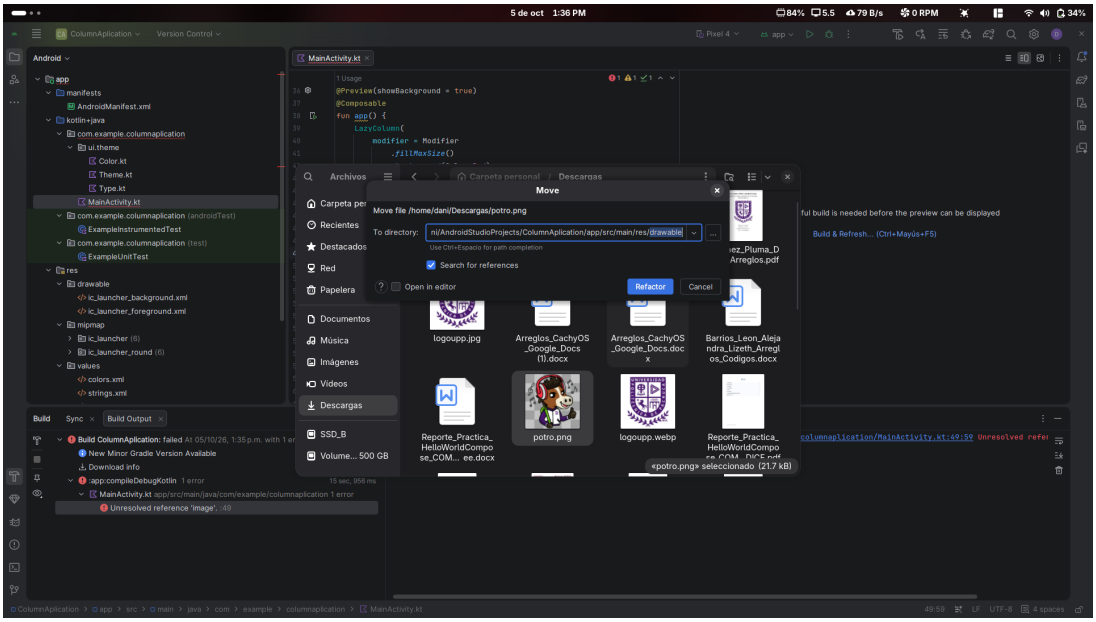


Ilustración 4: Importaciones y clase MainActivity con setContent.

3.4. Creación de la función composable

Se creó la función `app()`, identificada mediante la anotación `@Composable`, y se utilizó `@Preview` para visualizar la interfaz desde Android Studio. Como contenedor principal se utilizó una `LazyColumn` con `fillMaxSize()` y fondo rojo. Dentro de ella se agregó, en un `item`, un componente `Image` con ancho máximo y una altura de 400 dp.

```
@Composable
fun app() {
    LazyColumn(
        modifier = Modifier
            .fillMaxSize()
            .background(Color.Red)
    ) {
        item {
            Image(
                modifier = Modifier
                    .fillMaxWidth()
                    .height(400.dp),
                painter = painterResource(id = R.drawable.potro),
                contentDescription = "Logo"
            )
        }
```

Ilustración 5: Función composable app() con LazyColumn e Image.

3.5. Incorporación de los textos

Se agregaron tres componentes `Text` (“Danibellako”, “Suscribete” y “Hola”), cada uno en su propio `item`, con color blanco y alineación centrada. El texto principal utiliza un tamaño de fuente de 32 sp.

```
item {
    LazyRow(
        modifier = Modifier
            .fillMaxWidth()
            .padding( all = 16.dp),
        horizontalArrangement = Arrangement.SpaceBetween
    ) {
        item {
            Text(
                text = "Java",
                color = Color.White,
                modifier = Modifier.padding(end = 16.dp)
            )
            Text(
                text = "Kotlin",
                color = Color.White,
                modifier = Modifier.padding(end = 16.dp)
            )
            Text(
                text = "Suscríbete Suscríbete Suscríbete Suscríbete
                color = Color.White
```

Ilustración 6: Código de los componentes Text.

3.6. Implementación de LazyRow

Para incorporar la fila horizontal se agregó un `item` que contiene un `LazyRow`. A este se le aplicó `fillMaxWidth()` y `padding(16.dp)`, y se configuró `horizontalArrangement` con `Arrangement.SpaceBetween`. Dentro de la fila se colocaron los textos “Java”, “Kotlin” y un texto largo repetido (“Suscríbete”), con el objetivo de que el contenido superara el ancho de la pantalla y se pudiera comprobar el desplazamiento horizontal.

```
item {
    Text(
        text = "Danibellako",
        fontSize = 32.sp,
        color = Color.White,
        modifier = Modifier.fillMaxWidth(),
        textAlign = TextAlign.Center
    )
}

item {
    Text(
        text = "Suscribete",
        color = Color.White,
        modifier = Modifier.fillMaxWidth(),
        textAlign = TextAlign.Center
    )
}

item {
    Text(
        text = "Hola",
        color = Color.White,
        modifier = Modifier.fillMaxWidth(),
        textAlign = TextAlign.Center
    )
}
```

Ilustración 7: Código del LazyRow dentro de un item de la LazyColumn.

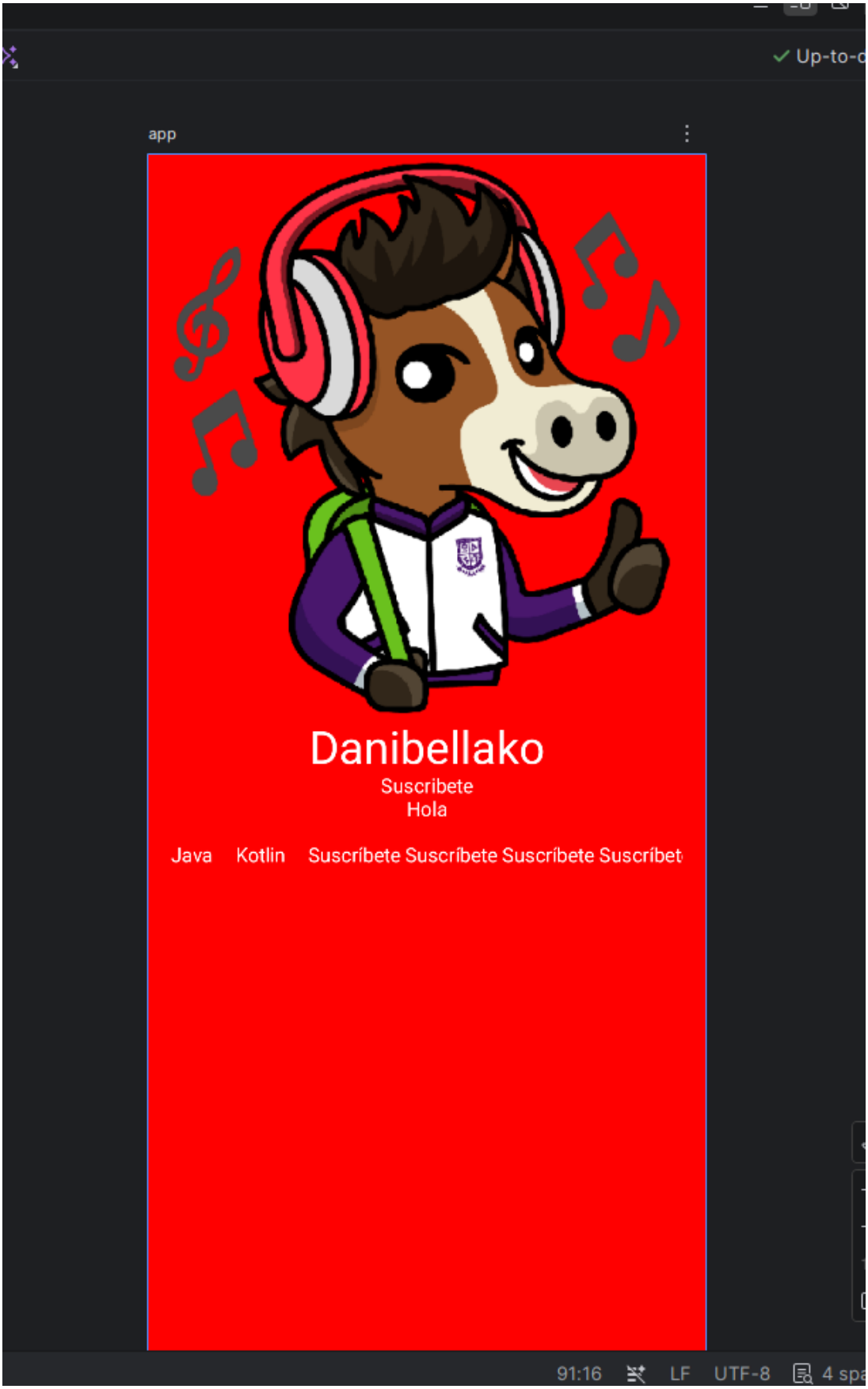


Ilustración 8: Vista previa de la pantalla con el LazyRow.

3.7. Pruebas de la aplicación

Se ejecutó la aplicación en el emulador para verificar el resultado. Se comprobó el desplazamiento vertical de la pantalla, que permite ver los elementos ubicados en la parte inferior.

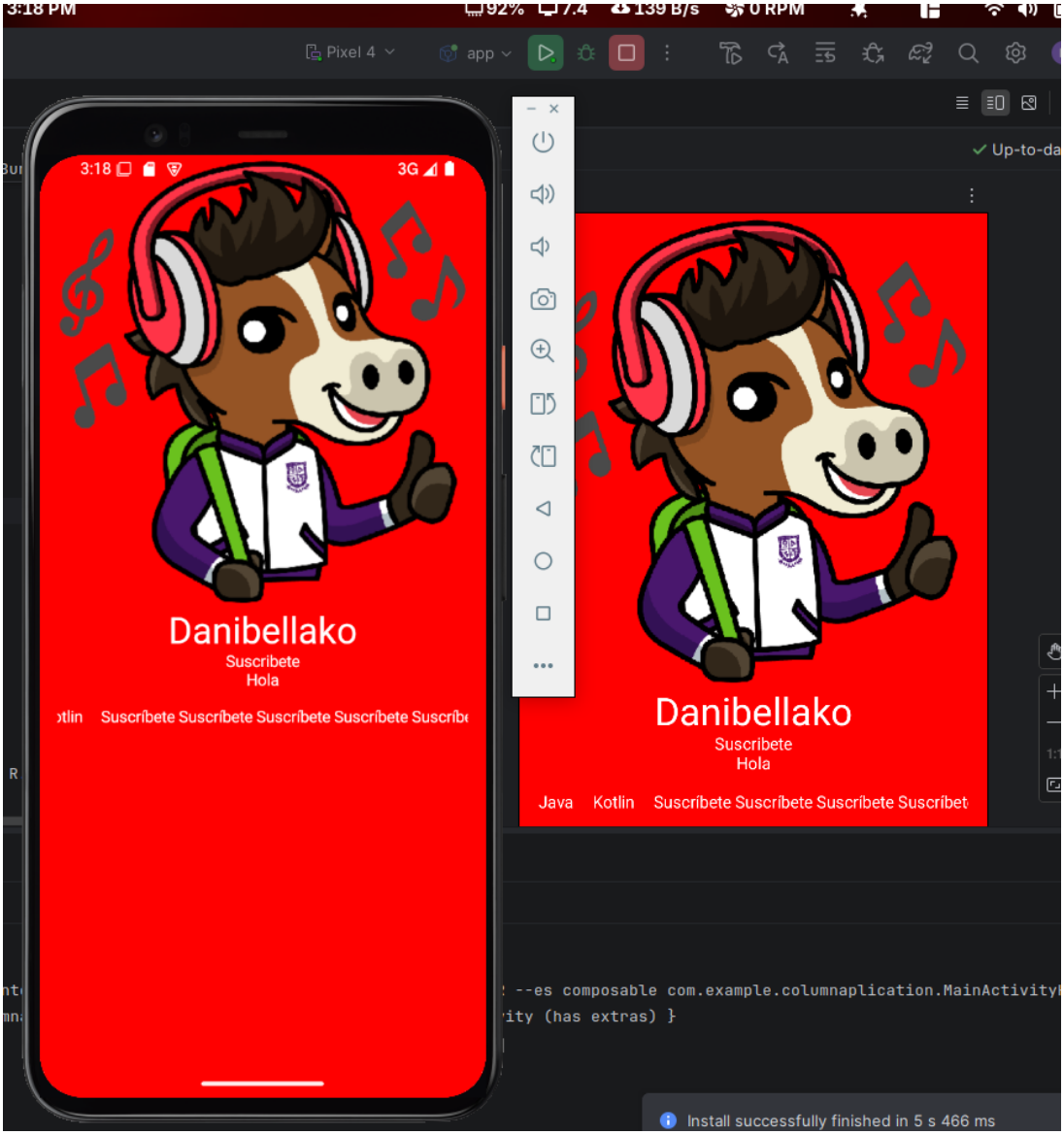


Ilustración 9: Desplazamiento vertical de la pantalla en el emulador.

También se comprobó el desplazamiento horizontal de la fila cuando el contenido supera el ancho visible. La interfaz final muestra la imagen, los tres textos centrados y la fila horizontal, todo dentro de una pantalla con desplazamiento vertical.

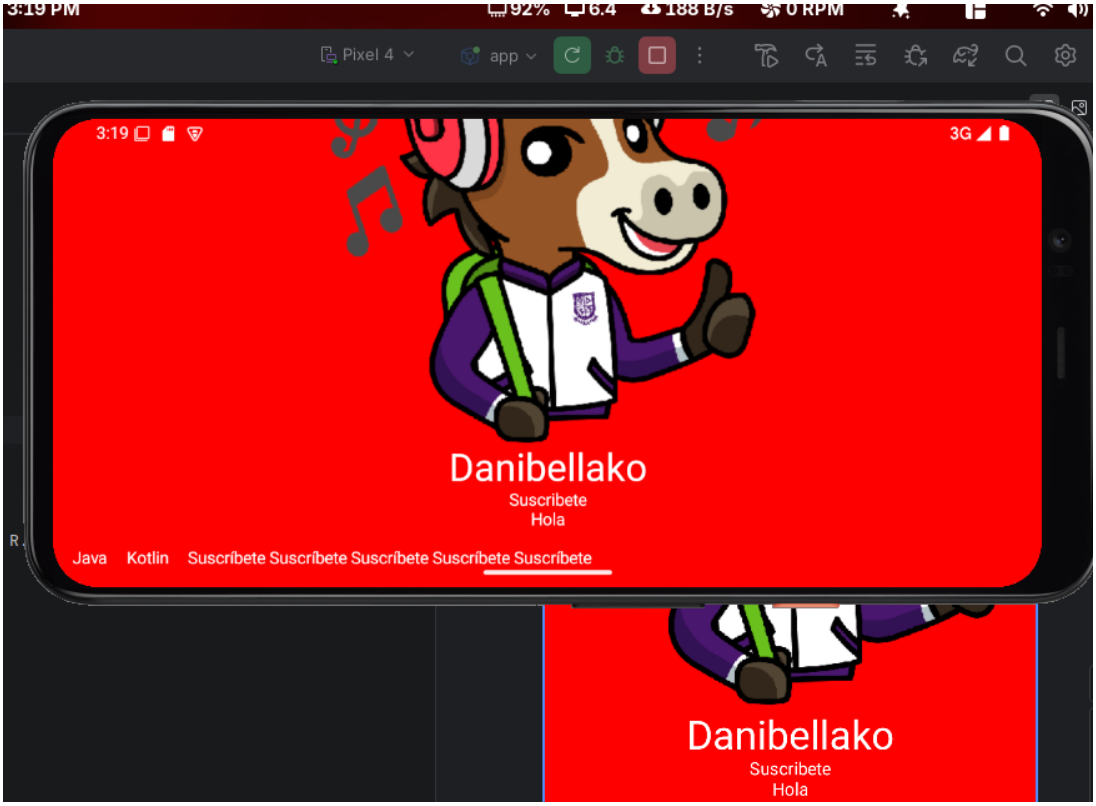


Ilustración 10: Desplazamiento horizontal del LazyRow y resultado final.

4. Análisis y argumentación

El desarrollo de la práctica permitió observar el funcionamiento de `LazyRow` y su integración con `LazyColumn`. Mientras que `Row` organiza elementos de forma horizontal sin incorporar desplazamiento, `LazyRow` añade desplazamiento horizontal y compone únicamente los elementos visibles (Google, s. f.-c).

También se comprobó que es posible combinar contenedores con direcciones de desplazamiento distintas. Al colocar el `LazyRow` dentro de un `item` de la `LazyColumn`, la pantalla se desplaza verticalmente y la fila se desplaza de forma independiente en horizontal.

Respecto a los modificadores, `fillMaxWidth()` permitió que la fila y los textos usaran todo el ancho disponible, `padding(16.dp)` separó la fila de los bordes y `height(400.dp)` fijó la altura de la imagen. En los textos, `TextAlign.Center` solo tiene efecto visible porque previamente se aplicó `fillMaxWidth()`: si la caja del texto fuera tan ancha como el propio texto, no habría espacio sobrante para centrarlo.

En cuanto a `Arrangement.SpaceBetween`, este reparte el espacio sobrante entre los elementos de la fila. Su efecto se aprecia cuando los elementos ocupan menos ancho que el contenedor; cuando el contenido supera el ancho visible, no hay espacio sobrante y la fila simplemente se desplaza.

La combinación de estos componentes permitió obtener una interfaz funcional y organizada.

5. Resultados

Como resultado de la práctica se obtuvo una aplicación Android desarrollada con Kotlin y Jetpack Compose.

La interfaz final contiene una imagen, tres elementos de texto centrados, una fila con textos desplazable horizontalmente y un fondo de color rojo. Los componentes se organizan verticalmente mediante `LazyColumn` y, dentro de ella, la fila horizontal se implementa con `LazyRow`.

También se comprobó el funcionamiento de los modificadores empleados para controlar el tamaño, ancho, altura, espaciado, fondo y alineación de los componentes.

6. Explicación del código

El código completo se encuentra en el Anexo A. A continuación se explica por bloques.

6.1. Paquete e importaciones

```
1  package com.example.columnaplication
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.activity.enableEdgeToEdge
7  import androidx.compose.foundation.Image
8  import androidx.compose.foundation.background
9  import androidx.compose.foundation.layout.Arrangement
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
13 import androidx.compose.foundation.layout.padding
14 import androidx.compose.foundation.lazy.LazyColumn
15 import androidx.compose.foundation.lazy.LazyRow
16 import androidx.compose.material3.Text
17 import androidx.compose.runtime.Composable
18 import androidx.compose.ui.Modifier
19 import androidx.compose.ui.graphics.Color
20 import androidx.compose.ui.res.painterResource
```

```
21                import androidx.compose.ui.text.style.TextAlign
22                import androidx.compose.ui.tooling.preview.Preview
23                import androidx.compose.ui.unit.dp
24                import androidx.compose.ui.unit.sp
25                import com.example.columnaplication.ui.theme.
    ColumnAplicationTheme
26
```

Descripción. La primera línea define el paquete del proyecto. Las importaciones traen las clases y funciones utilizadas: `Image` y `Text` para mostrar contenido; `LazyColumn` y `LazyRow` como listas con desplazamiento; `Arrangement` para distribuir elementos; `Modifier` y las funciones `fillMaxSize`, `fillMaxWidth`, `height`, `padding` y `background` para controlar tamaño y apariencia; `painterResource` para cargar la imagen; `TextAlign` para alinear texto; y `dp` y `sp` como unidades de dimensión y de fuente.

6.2. Actividad principal

```
1                 class MainActivity : ComponentActivity() {
2                         override fun onCreate(savedInstanceState: Bundle?) {
3                                 super.onCreate(savedInstanceState)
4                                 enableEdgeToEdge()
5                                 setContent {
6                                         ColumnAplicationTheme {
7                                                 app()
8                                         }
9                                 }
10                        }
11                }
12
```

Descripción. `MainActivity` es la pantalla que Android abre al iniciar la aplicación y hereda de `ComponentActivity`. Dentro de `onCreate()`, `enableEdgeToEdge()` permite dibujar la interfaz debajo de las barras del sistema y `setContent` establece la interfaz mediante Compose. El tema envuelve a la función `app()`, que es el composable principal.

6.3. Función composable y LazyColumn

```
1                 @Preview(showBackground = true)
2                 @Composable
3                 fun app() {
```

```
4                       LazyColumn(
5                       modifier = Modifier
6                       .fillMaxSize()
7                       .background(Color.Red)
8                       ) {
9                               // ... elementos de la lista (item)
10                      }
11              }
12
```

Descripción. `@Composable` identifica la función como parte de la interfaz y `@Preview` permite visualizarla en Android Studio. La `LazyColumn` es el contenedor principal y aporta desplazamiento vertical. Primero se aplica `fillMaxSize()` para ocupar toda la pantalla y después `background(Color.Red)` para pintar el fondo de rojo.

6.4. Imagen y textos

```
1               item {
2                       Image(
3                       modifier = Modifier
4                       .fillMaxWidth()
5                       .height(400.dp),
6                       painter = painterResource(id = R.drawable.potro),
7                       contentDescription = "Logo"
8                       )
9               }
10
11              item {
12                      Text(
13                      text = "Danibellako",
14                      fontSize = 32.sp,
15                      color = Color.White,
16                      modifier = Modifier.fillMaxWidth(),
17                      textAlign = TextAlign.Center
18                      )
19              }
20
```

Descripción. Cada `item` es un elemento de la lista vertical. La imagen ocupa todo el ancho y tiene 400 dp de alto; `painterResource` carga el recurso `potro` y `contentDescription` aporta accesibilidad. El texto principal usa 32 sp, color blanco y

ocupa todo el ancho para que `TextAlign.Center` pueda centrarlo. Los textos "Suscribete" y "Hola" siguen el mismo patrón, sin definir tamaño de fuente.

6.5. LazyRow

```
                item {
                        LazyRow(
                        modifier = Modifier
                        .fillMaxWidth()
                        .padding(16.dp),
                        horizontalArrangement = Arrangement.SpaceBetween
                        ) {
                                item {
                                        Text(
                                        text = "Java",
                                        color = Color.White,
                                        modifier = Modifier.padding(end = 16.dp)
                                        )
                                        Text(
                                        text = "Kotlin",
                                        color = Color.White,
                                        modifier = Modifier.padding(end = 16.dp)
                                        )
                                        Text(
                                        text = "Suscríbete Suscríbete Suscríbete
    Suscríbete Suscríbete",
                                        color = Color.White
                                        )
                                }
                        }
                }

```

Descripción. El `LazyRow` se coloca dentro de un `item` de la `LazyColumn`. Ocupa todo el ancho, deja 16 dp de espacio alrededor mediante `padding(16.dp)` y utiliza `Arrangement.SpaceBetween` como distribución horizontal. Los textos "Java" y "Kotlin" dejan 16 dp a su derecha con `padding(end = 16.dp)`. El tercer texto es largo a propósito para que el contenido supere el ancho de la pantalla y se pueda comprobar el desplazamiento horizontal.

7. Elementos principales utilizados

Tabla 1: Elementos principales utilizados en la práctica

Elemento	Función
`MainActivity`	Actividad principal de la aplicación.
`setContent`	Establece el contenido de la interfaz mediante Compose.
`@Composable`	Identifica una función que construye elementos de interfaz.
`@Preview`	Permite visualizar una vista previa de la interfaz.
`LazyColumn`	Organiza elementos verticalmente y permite desplazamiento vertical.
`LazyRow`	Organiza elementos horizontalmente y permite desplazamiento horizontal.
`item{}`	Agrega un elemento dentro de una lista lazy.
`Image`	Permite mostrar una imagen.
`Text`	Permite mostrar texto.
`Modifier`	Permite modificar características de los componentes.
`fillMaxSize()`	Permite ocupar el espacio disponible.
`fillMaxWidth()`	Permite ocupar el ancho disponible.
`height()`	Establece una altura determinada.
`padding()`	Agrega espacio alrededor del componente.
`background()`	Establece el color de fondo.
`Arrangement`	Define la distribución de los elementos en el eje principal.
`TextAlign.Center`	Centra el contenido del texto.

8. Conclusiones

8.1. Aportes y limitaciones

La realización de esta práctica permitió comprender cómo se organiza contenido de forma horizontal con desplazamiento en Jetpack Compose mediante `LazyRow`, y cómo integrarlo dentro de una `LazyColumn`. También se consolidó el uso de modificadores, de `Arrangement` y de bloques `item{}`.

Entre las limitaciones se encuentra que la interfaz utiliza contenido estático y

textos escritos directamente en el código, sin recursos de cadenas. Tampoco se trabajaron los márgenes de las barras del sistema al usar `enableEdgeToEdge()`, y las pruebas no evaluaron distintos tamaños de pantalla.

8.2. Recomendaciones y proyecciones

Se recomienda generar los elementos de la fila a partir de una lista de datos con `items()`, mover los textos a `res/values/strings.xml`, manejar los márgenes del sistema con `Scaffold` y probar la interfaz en diferentes tamaños de pantalla.

9. Trabajo futuro

Como continuación de esta práctica se pueden implementar las siguientes mejoras:

- Generar los elementos del `LazyRow` a partir de una lista de datos.
- Sustituir los textos por tarjetas o botones.
- Incorporar interacción con el usuario mediante eventos de clic.
- Utilizar recursos de cadenas para facilitar el mantenimiento.
- Mejorar la adaptación de la interfaz a diferentes tamaños de pantalla.
- Explorar otros componentes de Jetpack Compose.

10. Anexos

10.1. Anexo A. Código completo

```
1  package com.example.columnaplication
2
3  import android.os.Bundle
4  import androidx.activity.ComponentActivity
5  import androidx.activity.compose.setContent
6  import androidx.activity.enableEdgeToEdge
7  import androidx.compose.foundation.Image
8  import androidx.compose.foundation.background
9  import androidx.compose.foundation.layout.Arrangement
10 import androidx.compose.foundation.layout.fillMaxSize
11 import androidx.compose.foundation.layout.fillMaxWidth
12 import androidx.compose.foundation.layout.height
```

```
13              import androidx.compose.foundation.layout.padding
14              import androidx.compose.foundation.lazy.LazyColumn
15              import androidx.compose.foundation.lazy.LazyRow
16              import androidx.compose.material3.Text
17              import androidx.compose.runtime.Composable
18              import androidx.compose.ui.Modifier
19              import androidx.compose.ui.graphics.Color
20              import androidx.compose.ui.res.painterResource
21              import androidx.compose.ui.text.style.TextAlign
22              import androidx.compose.ui.tooling.preview.Preview
23              import androidx.compose.ui.unit.dp
24              import androidx.compose.ui.unit.sp
25              import com.example.columnaplication.ui.theme.
    ColumnAplicationTheme
26
27              class MainActivity : ComponentActivity() {
28                      override fun onCreate(savedInstanceState: Bundle?) {
29                              super.onCreate(savedInstanceState)
30                              enableEdgeToEdge()
31                              setContent {
32                                      ColumnAplicationTheme {
33                                              app()
34                                      }
35                              }
36                      }
37              }
38
39              @Preview(showBackground = true)
40              @Composable
41              fun app() {
42                      LazyColumn(
43                      modifier = Modifier
44                      .fillMaxSize()
45                      .background(Color.Red)
46                      ) {
47                              item {
48                                      Image(
49                                      modifier = Modifier
50                                      .fillMaxWidth()
51                                      .height(400.dp),
```

```
52                                      painter = painterResource(id = R.
    drawable.potro),
53                                      contentDescription = "Logo"
54                                      )
55                                  }
56
57                                  item {
58                                      Text(
59                                      text = "Danibellako",
60                                      fontSize = 32.sp,
61                                      color = Color.White,
62                                      modifier = Modifier.fillMaxWidth(),
63                                      textAlign = TextAlign.Center
64                                      )
65                                  }
66
67                                  item {
68                                      Text(
69                                      text = "Suscribete",
70                                      color = Color.White,
71                                      modifier = Modifier.fillMaxWidth(),
72                                      textAlign = TextAlign.Center
73                                      )
74                                  }
75
76                                  item {
77                                      Text(
78                                      text = "Hola",
79                                      color = Color.White,
80                                      modifier = Modifier.fillMaxWidth(),
81                                      textAlign = TextAlign.Center
82                                      )
83                                  }
84
85                                  item {
86                                      LazyRow(
87                                      modifier = Modifier
88                                      .fillMaxWidth()
89                                      .padding(16.dp),
90                                      horizontalArrangement = Arrangement.
    SpaceBetween
```

```
91                                      ) {
92                                              item {
93                                                      Text(
94                                                      text = "Java",
95                                                      color = Color.White,
96                                                      modifier = Modifier.
    padding(end = 16.dp)
97                                                      )
98                                                      Text(
99                                                      text = "Kotlin",
100                                                     color = Color.White,
101                                                     modifier = Modifier.
    padding(end = 16.dp)
102                                                     )
103                                                     Text(
104                                                     text = "Suscríbete Suscr
    íbete Suscríbete Suscríbete Suscríbete",
105                                                     color = Color.White
106                                                     )
107                                             }
108                                     }
109                                 }
110                             }
111                 }
112
```

10.2. Anexo B. Evidencias adicionales

Las diez ilustraciones incluidas en el desarrollo del documento representan las principales etapas realizadas durante la práctica. Las imágenes utilizadas corresponden a los archivos `1.png` a `10.png`.

11. Referencias

- AristiDevs. ([2021]). *[FILAS (ROW) en JETPACK COMPOSE - Curso Jetpack Compose desde cero - Tutorial Android COMPOSE [2021]]* [Video]. YouTube. https://youtu.be/tIhioKYvtGE
- Google. (s. f.-a). *Jetpack Compose*. Android Developers. https://developer.android.com/compose

- Google. (s. f.-b). *Fundamentos de los diseños de Compose*. Android Developers. https://developer.android.com/develop/ui/compose/layouts/basics
- Google. (s. f.-c). *Listas y cuadrículas en Compose*. Android Developers. https://developer.android.com/develop/ui/compose/lists
- JetBrains. (s. f.). *Kotlin documentation*. Kotlin. https://kotlinlang.org/docs/home.html
- Universidad Politécnica de Pachuca. (s. f.). *Guía de estructura y desarrollo de documentos académicos*. Material proporcionado para la asignatura.